

Click-to-Source

Technical Implementation Approaches — Comparison & Decision Guide

Prepared for Punith P | August 2026

This document lays out every realistic way to technically build the Click-to-Source tool, explained in plain terms with simple visual flows, so the best option can be picked deliberately instead of by default. Each approach solves the same core problem — click a rendered feature, resolve it to source, edit it — using a different piece of infrastructure to bridge the browser and the file system.

Primary Objective

Validate whether developers naturally debug procedural scenes by interacting with rendered output.

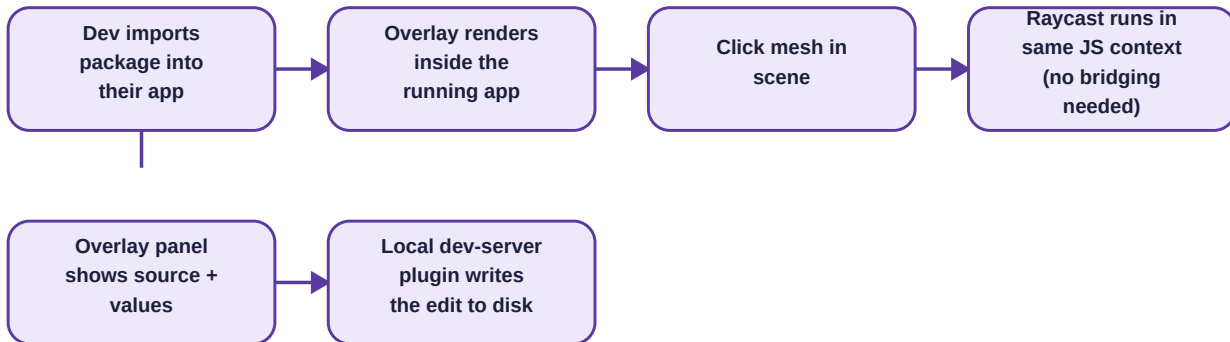
Every approach, comparison, and recommendation in this document exists to answer that one question as cheaply and quickly as possible — not to pick a permanent architecture on day one.

The One Constraint That Shapes Everything

A web page running in a browser cannot read or write files on your computer — this is a security boundary browsers enforce, not a limitation of this project. That means no matter which approach is chosen, something outside the browser tab has to handle reading the source file and writing edits back to disk. The five approaches below are really five different answers to **"what plays that outside role?"**

Approach 1 — In-App Debug Overlay (NPM Package)

A small package (e.g. `click-to-source-r3f`) that a developer imports directly into their own React Three Fiber app, similar to how React Query Devtools or Leva work. It renders an overlay panel inside the app itself, only active in development mode.



Pros

- Simplest to build — no browser extension APIs, no cross-context messaging, no permissions to request.
- Direct access to the live scene since the overlay runs inside the same app instance.
- Easiest to test on your own Procedural World Generator immediately.

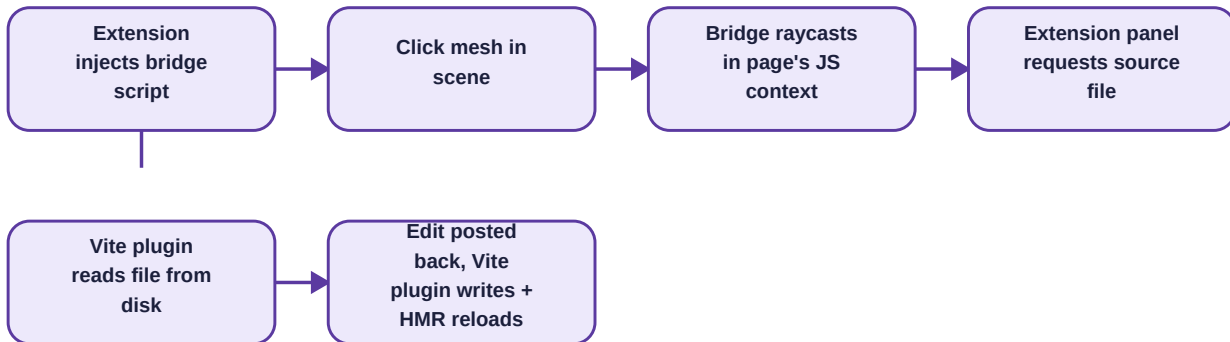
Cons

- Only works on apps that explicitly install the package — not a general "point at any site" tool.
- Every project has to add a dependency, which is a small adoption cost.

A note on treating this as "just the MVP": it's tempting to view Approach 1 as a disposable stepping stone toward a browser extension or IDE plugin, but that's an assumption worth questioning rather than a conclusion. Leva and React Query Devtools both succeeded purely as npm packages — though it's worth being even-handed here, since React DevTools and Redux DevTools, arguably the two most successful tools in this exact space, both shipped as browser extensions instead. The precedent genuinely cuts both ways, which is itself the argument for not pre-committing: build Approach 1, see whether it feels complete on its own, and let real usage — not category precedent — decide whether it needs to become something else.

Approach 2 — Browser Extension + Local Dev-Server Bridge

A Chrome/Firefox extension injects a script into the page to reach the live Three.js scene, and talks to a companion Vite plugin running on your local dev server to read and write source files. This is the architecture Needle Inspector uses.



Pros

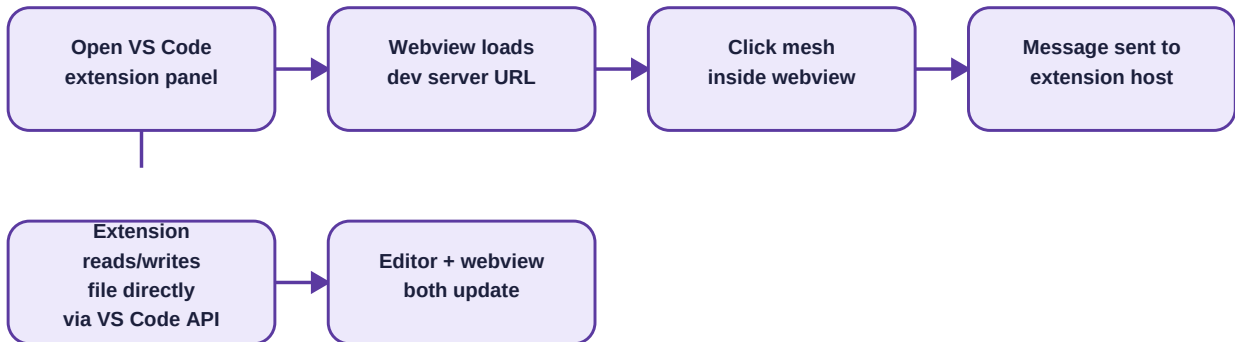
- No per-project dependency needed beyond the small Vite plugin — works across any project once installed.
- Familiar distribution model (Chrome Web Store) with real precedent (Needle Inspector, React DevTools).

Cons

- Most moving parts of all approaches — extension permissions, cross-world messaging, a separate local server plugin.
- Extra browser-security friction (content security policies, isolated worlds) to work through.
- Worth being explicit that this is more than a phrase like "most moving parts" implies: Manifest V3 service-worker constraints, isolated-world message passing, CSP restrictions, permission scopes, and cross-browser differences (Chrome vs. Firefox) each carry their own learning curve. None are individually hard, but stacked together they can quietly become the project on their own if underestimated.

Approach 3 — VS Code Extension with Webview

Everything lives inside VS Code. A webview panel loads the running scene (pointing at your local dev server URL, e.g. localhost:5173) inside an embedded browser view. Because it's a VS Code extension, it already has full file system access — no separate plugin/server needed for reading or writing source.



Pros

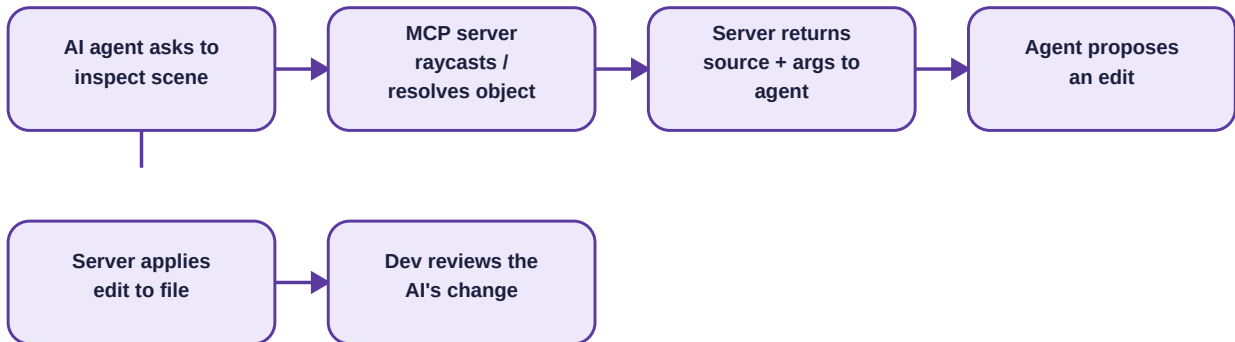
- File access is built in — no custom server-side plugin needed for reading/writing.
- Matches where most developers already work: **VS Code holds roughly 76% IDE usage share**, so this meets people where they are.
- Source and 3D view sit side by side in one window, which is a clean UX.

Cons

- VS Code webviews are sandboxed iframes — getting a full WebGL/Three.js scene running smoothly inside one needs care.
- Only reaches VS Code users (the large majority, but not everyone).
- Extension API has a steeper learning curve than a plain npm package.

Approach 4 — MCP Server / AI Agent Tool

Instead of a human-facing UI, expose "click-inspect" and "resolve-to-source" as tools an AI agent (Claude Desktop, Cursor, etc.) can call directly via the Model Context Protocol — the same approach the Threejs Devtools MCP server uses.



Pros

- Turns your original token-saving thesis into the literal mechanism — the AI gets exact context for free instead of guessing.
- No custom UI to design or maintain at all.

Cons

- Removes the satisfying "click and see it instantly" human experience — everything happens through an AI chat step.
- Hardest to demo visually for a portfolio/LinkedIn audience, since there's no interactive UI to show.

Approach 5 — Standalone Electron App

A full desktop app that bundles its own browser view and file system access together, similar to how Figma Desktop or Postman work. You'd load the project's dev server inside it and manage everything from one packaged application.

Pros

- Full control over the environment — no browser extension or IDE API limitations at all.

Cons

- By far the most work to build and package (Electron adds real overhead on a constrained 2-core/16GB machine).
- Asks developers to open a whole separate app just to debug — the highest adoption cost of any option.
- Not recommended at this stage — listed for completeness only.

Side-by-Side Comparison

Approach	Build Effort	Works Across Any Project?	Meets Devs Where They Work?	Best Fit For
1. NPM Overlay Package	Low	No — opt-in per project	Inside the app itself	MVP / v0.1 — proving the core loop fast
2. Browser Extension + Vite Bridge	High	Yes, once installed	Chrome/browser toolbar	v1 public distribution, once proven
3. VS Code Extension + Webview	Medium-High	Yes, per opened project	VS Code (~76% of devs)	Strong v1 alternative to Approach 2
4. MCP Server / AI Tool	Medium	Yes, via any MCP-compatible agent	Inside AI coding assistants	Stage 5 add-on (AI handoff), not the core UI
5. Standalone Electron App	Very High	Yes	Nowhere — new app to adopt	Not recommended currently

Recommendation

One part of this is settled; one part deliberately isn't:

- **Start with Approach 1 (NPM overlay package) — this part is settled.** Lowest effort, fastest path to a working demo, and it directly matches the MVP scope already defined (Stage 3: one demo scene, manual tagging, no AI). This is how the core mechanism gets proven in weeks, not months. It should not be treated as automatically disposable — if it turns out to feel complete on its own, there's no rule saying it has to become something else.
- **What comes after Approach 1 is deliberately left as a branch, not a fixed step 2.** There's a real argument for Approach 3 (VS Code extension) next — it reaches the ~76% of developers already on VS Code with moderate added effort. There's an equally real argument for Approach 2 (browser extension) next — if the overlay from Approach 1 already feels good, a browser extension removes the biggest remaining friction (installing a dependency) more directly than an IDE plugin does. Both are legitimate; picking between them before real usage data exists would be guessing. The right move is to validate with a handful of actual users after Approach 1 ships, and let that answer decide 2 vs. 3 — not precedent, not this document.
- **Add Approach 4 (MCP server) as a bolt-on, not a replacement** — this is precisely Stage 5 from the staged plan ("AI handoff"), and it can sit on top of whichever UI approach is chosen without rearchitecting anything.
- **Skip Approach 5 (Electron)** for now — the effort-to-benefit ratio is the worst of all five, especially given hardware constraints.

In short: **Approach 1 to prove it works, then validate with real users to decide between Approach 3 (reach VS Code's ~76% of developers) and Approach 2 (remove friction further with a browser extension), with Approach 4 layered on top once there's something worth feeding to AI.**

Decision Criteria Matrix

The recommendation above is reasoned in prose. This table makes the same reasoning explicit and scorable, so future decisions (e.g. choosing between Approach 2 and 3 after validation) can reuse this structure instead of re-arguing it from scratch. Scored High / Medium / Low per criterion.

Approach	Fastest MVP	Low Maintenance	Best Adoption	Easy Demo	Resume Value	OSS Potential	Low Tech Risk
1. NPM Overlay	High	High	Medium	High	Medium	High	High
2. Browser Ext.	Low	Medium	High	Medium	High	High	Low
3. VS Code Ext.	Medium	Medium	High	Medium	High	Medium	Medium
4. MCP Server	Medium	High	Medium	Low	High	Medium	Medium
5. Electron	Low	Low	Low	Medium	Medium	Low	Low

Reading the matrix: Approach 1 wins or ties on nearly every criterion except adoption reach, which is exactly why it's the settled starting point rather than one option among five. Approaches 2 and 3 trade off almost symmetrically — 2 wins on adoption and resume novelty but loses badly on technical risk; 3 is the more balanced, lower-risk middle option. This mirrors the branch left open in the Recommendation section, rather than resolving it — the matrix is a tool for that future decision, not a substitute for the user validation it depends on.

Platform Potential (Long-Term Note — Not Part of the Near-Term Plan)

Every approach in this document relies on the same underlying chain: **Runtime Object** → **Metadata** → **Source** → **Arguments**. That chain is general-purpose — once it exists, it could plausibly support things beyond Click-to-Source itself: a performance profiler tied to exact generator calls, a live parameter editor, richer AI context provision, a runtime call graph, a more general visual debugger, or a change-history view over generated content.

This is worth writing down, but worth treating carefully. The original project idea sprawled once before — a code editor grew to include a git UI, a multi-language runner, and a multi-engine plugin — before being scoped down to Click-to-Source specifically. "This could become a platform" is structurally the same kind of pull, and it's worth naming that explicitly so it doesn't quietly widen the MVP. The discipline here is: **build Click-to-Source well first: whether the underlying instrumentation is actually worth generalizing into a platform is a question to earn after that, not assume before it.**